

5장 part 1. (5.1~5.2)

5.1 합성곱 신경망 (CNN)

5.1.1 합성곱층의 필요성

- 이미지나 영상 처리에 유용
- 이미지를 펼쳐서 분석하면 데이터의 공간적 구조를 무시하는 문제 → 방지하기 위해 합성곱층 도입

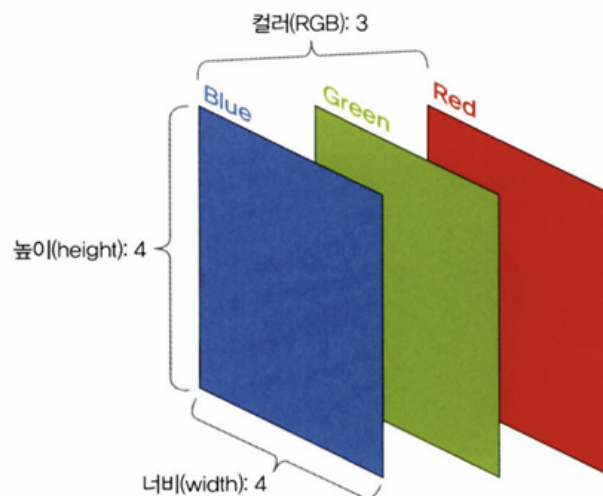
5.1.2 합성곱 신경망의 구조

- 다차원 배열 처리에 특화됨.
- 합성곱층과 풀링층을 거치며 이미지의 주요 벡터를 추출하고, 추출된 벡터들은 완전연결층을 거치며 1차원 벡터로 변환되며, 출력층에서 활성화 함수를 사용하여 최종 결과 출력

1. 입력층

- 입력 이미지 데이터가 최초로 거치게 되는 계층
- 이미지 : 높이, 너비, 채널 값을 가짐
 - 채널 : 이미지가 grayscale이면 1, RGB(컬러)이면 3
 - (4,4,3)

▼ 그림 5-3 채널



2. 합성곱층

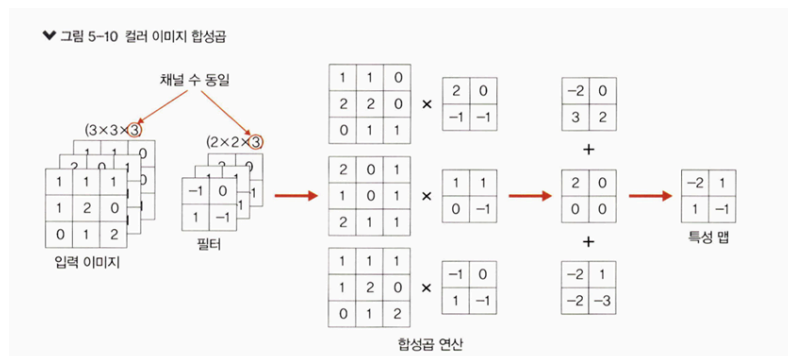
- 입력 데이터에서 특성을 추출하는 역할

- 특성 추출 : 커널/필터를 이용해 이미지의 모든 영역을 훑음 → 특성 맵
- stride : 지정된 간격
- 커널 : 3*3, 5*5
- 스트라이드가 1일 때 이동 (그레이스케일)



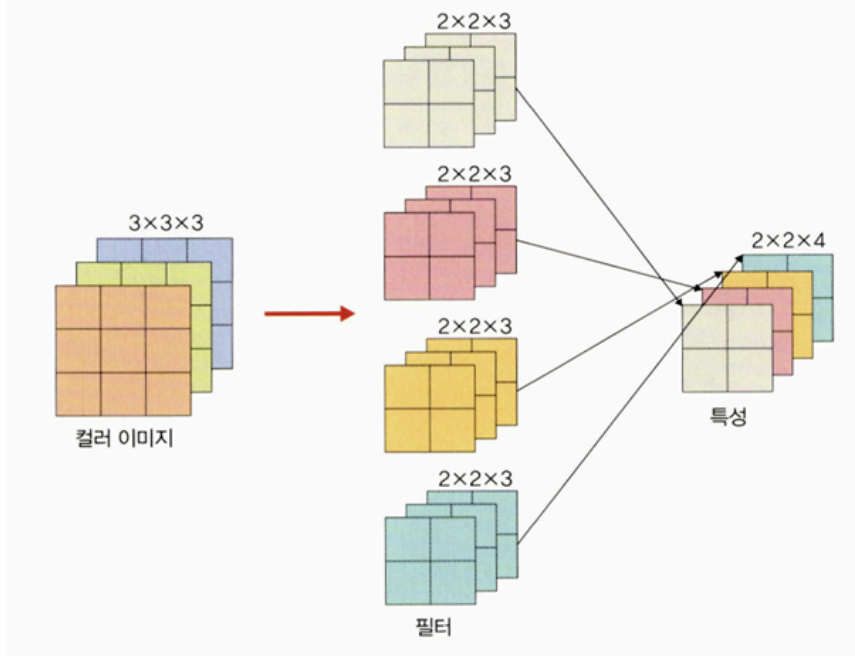
(6,6,1) → (4,4,1) 특성 맵으로 줄어듦

- RGB의 이동 (필터 채널 3, RGB 각각에 서로 다른 가중치로 합성곱을 적용 후 결과를 더함)



- 필터가 2개 이상인 합성곱 → 필터 각각은 특성 추출 결과의 채널이 됨

▼ 그림 5-11 필터가 2 이상인 합성곱



입력 데이터 $W * H * D$ (W : 가로, H : 세로, D : 채널 또는 깊이)

하이퍼파라미터

- 필터 개수 K , 필터 크기 F , 스트라이드 S , 패딩 P

출력 데이터

- $W_2 = (W_1 - F + 2P) / S + 1$
- $H_2 = (H_1 - F + 2P) / S + 1$
- $D = K$

3. 풀링층

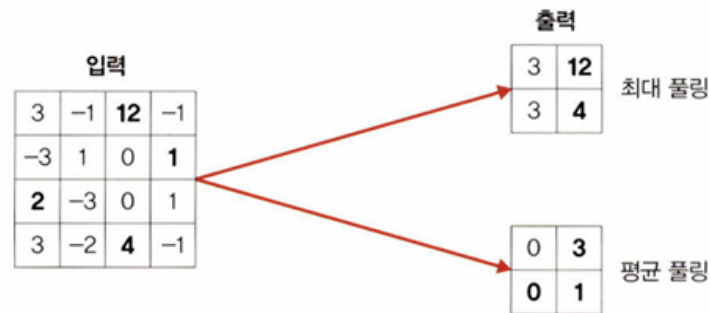
특성 맵의 차원을 다운 샘플링 → 연산량을 감소시키고 주요한 특성 벡터 추출 → 효과적인 학습

- Max pooling : 대상 영역에서 최댓값 추출 → 가장 많이 쓰임

▼ 그림 5-16 네 번째 최대 풀링 과정



- Average pooling : 대상 영역에서 평균 반환



입력 데이터 : $W1 * H1 * D1$

하이퍼파라미터

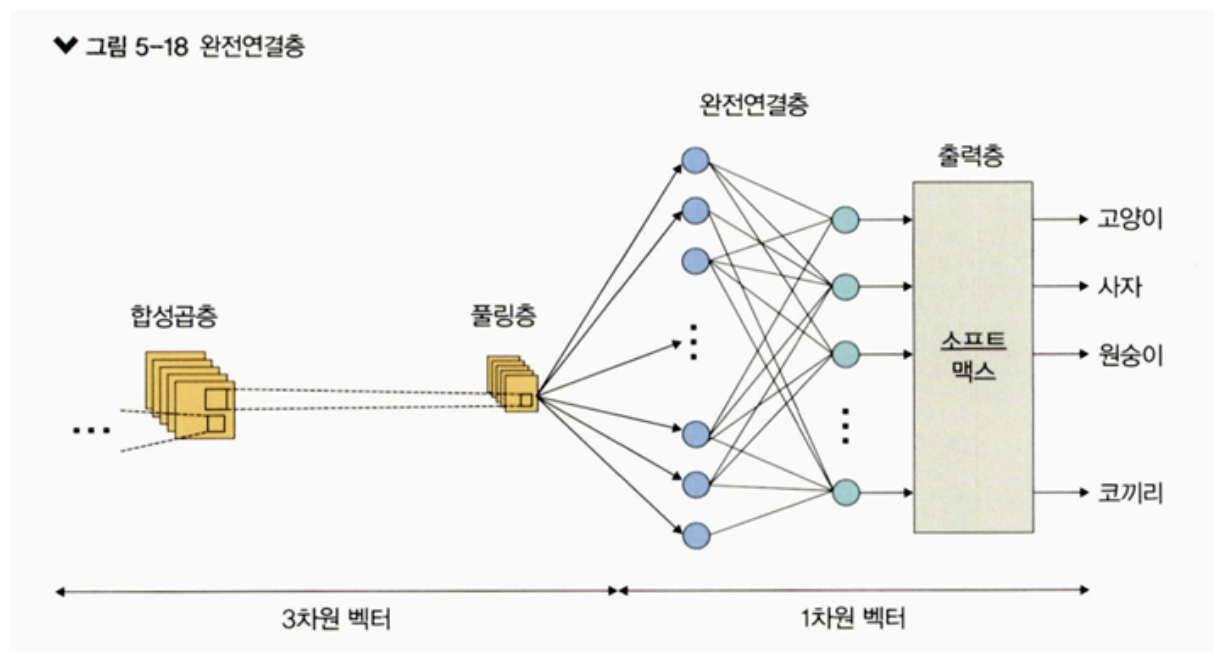
- 필터 크기 : F, 스트라이드 S

출력 데이터

- $W2 = (W1-F)/S+1$, $H2 = (H1-F)/S+1$, $D2 = D1$

4. 완전연결층

- 이미지가 3차원 벡터에서 1차원 벡터로 펼쳐짐 (Flatten)



5. 출력층

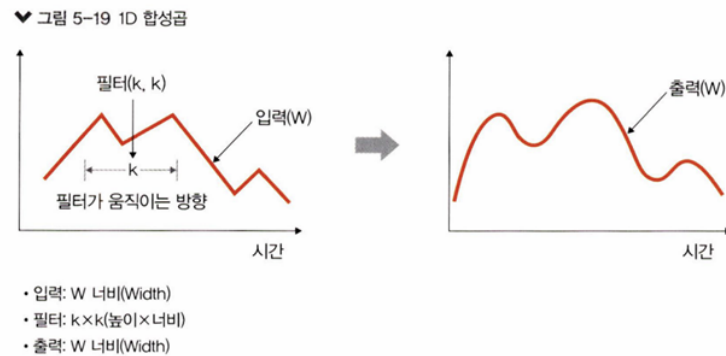
소프트맥스 활성화 함수 사용 : 입력받은 값을 0~1 사이의 값으로 출력 → 마지막 출력층에서 이미지가 각 레이블에 속할 확률값이 출력되고, 가장 높은 확률 값을 갖는 레이블이 최종 선정됨

5.1.3 1D,2D,3D 합성곱

1. 1D 합성곱

필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱

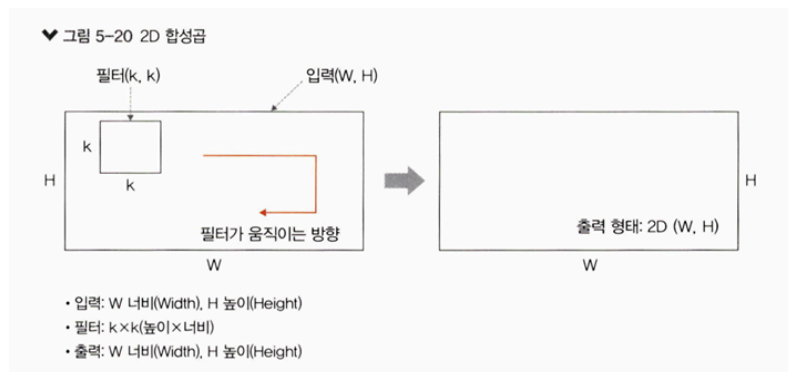
입력(W)과 필터(k)에 대한 출력은 $W \rightarrow$ 출력 형태는 1D 배열



2. 2D 합성곱

방향 2개로 움직이는 형태

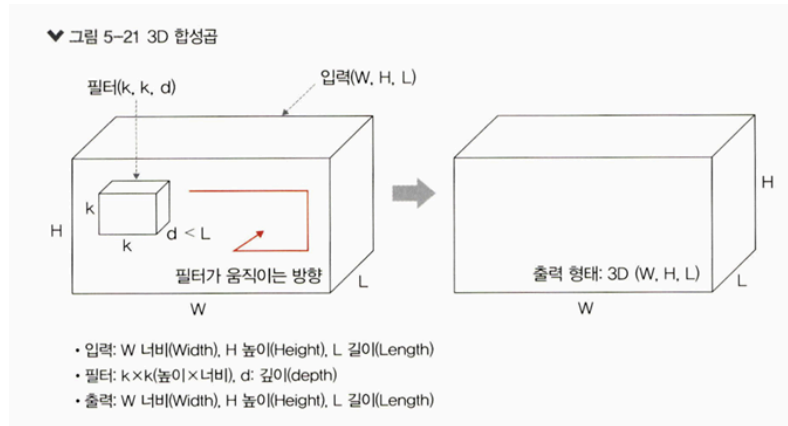
입력(W,H)과 필터(k,k)에 대한 출력은 (W,H) $\rightarrow$ 출력 형태는 2D 행렬



3. 3D 합성곱

필터가 움직이는 방향이 3개

입력(W,H,L)에 대해 필터(k,k,d)를 적용 $\rightarrow$ 출력으로 (W,H,L) 인 3D 형태이며, $d < L$ 유지

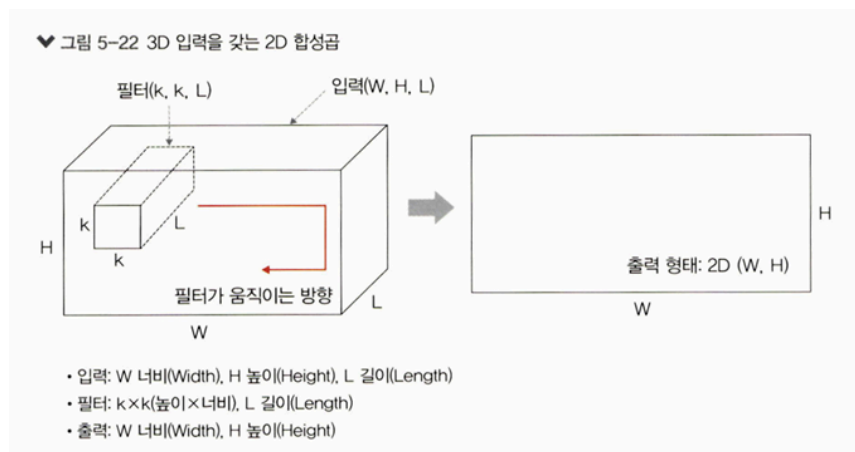


4. 3D 입력을 갖는 2D 합성곱

→ 입력이 3D 형태지만 출력 형태가 2D 행렬 : 필터에 대한 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문

입력 (W, H, L)에 필터(k, k, L) 적용 → 출력 (W, H)

ex) LeNet-5, VGG



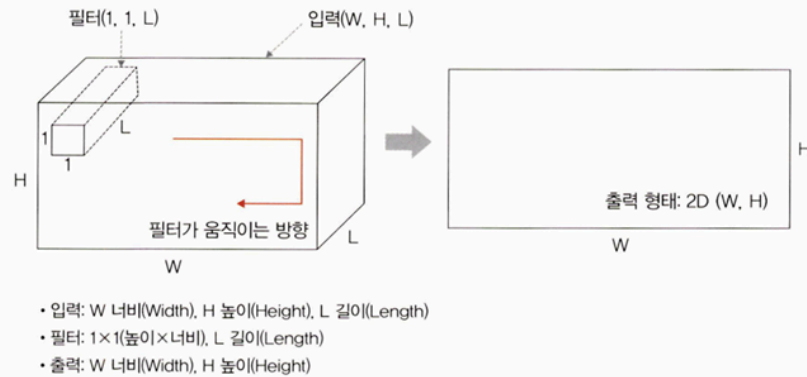
5. 1*1 합성곱

3D 형태로 입력됨 : 입력(W, H, L)에 필터($1, 1, L$) 적용 → 출력 (W, H)

1*1 합성곱에서 채널 수를 조정해 연산량이 감소됨

ex) GoogLeNet

▼ 그림 5-23 1×1 합성곱



5.2 합성곱 신경망 맛보기

fashion mnist 데이터셋으로 구현

- torchvision.datasets은 torch.utils.data.Dataset의 하위 클래스 : 다양한 데이터셋 포함
 - FashionMNIST 내려받을 위치 지정
 - download : TRUE → 해당 데이터셋 존재 확인 후 다운
 - transform : 이미지를 텐서(0~1)로 변경
- torch.utils.data.DataLoader() → 원하는 크기의 배치 단위로 데이터 다운 및 shuffle 가능
- batch_size : 데이터를 배치로 묶어줌

```
labels_map = {0 : 'T-Shirt', 1 : 'Trouser', 2 : 'Pullover', 3 : 'Dress', 4 : 'Coat', 5 : 'Sandal', 6 : 'Shirt', 7 : 'Sneaker', 8 : 'Bag', 9 : 'Ankle Boot'}
```

```
fig = plt.figure(figsize=(8,8));
columns = 4;
rows = 5;
for i in range(1, columns*rows +1):
    img_xy = np.random.randint(len(train_dataset));
    img = train_dataset[img_xy][0][0,:,:]
    fig.add_subplot(rows, columns, i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
    plt.imshow(img, cmap='gray')
plt.show()
```

- `np.random.randint()` : 무작위 표본을 추출할 때 사용

```
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256)
        self.drop = nn.Dropout2d(0.25)
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data):
        out = input_data.view(-1, 784)
        out = F.relu(self.fc1(out))
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

- class 형태의 모델은 `torch.nn.Module` 상속받음
- `__init__()` : 객체가 갖는 속성값 초기화
- `nn` : 딥러닝 모델 구성에 필요한 모듈이 모여있는 패키지
- `Linear` : 단순 선형회귀모델 만들 때 필요
 - `in_features` : 입력의 크기
 - `out_features` : 출력의 크기
- `torch.nn.Dropout(p)` : p 만큼의 비율로 텐서 값이 0이 되고, 0이 되지 않는 값들은 기존 값에 $(1/(1-p))$ 만큼 곱해져 커짐
- `forward()` : 학습 데이터를 입력받아서 순전파 학습 진행
- `view` : 넘파이의 `reshape`와 같은 역할로 텐서의 크기를 변경
- 활성화 함수 지정
 - `F.relu()` : `forward()` 함수에서 정의
 - `nn.ReLU()` : `init()` 함수에서 정의
 - 차이 : 사용하는 위치
- 모델 학습 전 손실함수, 학습률, 옵티마이저 정의

```
learning_rate = 0.001;
model = FashionDNN();
model.to(device)
```



```

criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
print(model)

```

- 옵티마이저를 위한 경사 하강법은 Adam, 학습률은 0.01

```

num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)

            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()

```

```

        total += len(labels)

    accuracy = correct * 100 / total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}%".format
              (count, loss.data, accuracy))

```

- 모델이 데이터를 처리하기 위해서는 모델과 데이터가 동일한 장치에 있어야 함
- Autograd : 자동미분을 수행하는 파이토치의 핵심 패키지 → 자동 미분에 대한 값을 저장하기 위해 tape 사용
 - 순전파 단계 : 테이프는 수행하는 모든 연산 저장
 - 역전파 단계 : 저장된 값 꺼내서 사용
- 분류 문제에 대한 정확도는 전체 예측에 대한 정확한 예측의 비율로 표현 가능
- 분류 문제에서 클래스가 3개 이상일 때
 - 정확도가 높더라도, 그 정확도 값이 모든 클래스가 동등하게 고려된 것인지, 특정 클래스의 분류가 높았던 것인지 알 수 없음
 - 모든 데이터가 하나의 클래스에 속한 경우일수도 있으니 확인 필요

합성곱 신경망

```

class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=
3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size
=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),

```

```

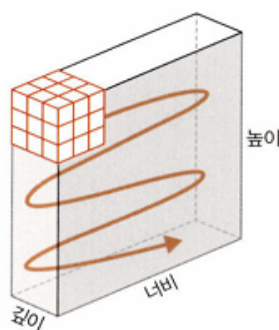
        nn.MaxPool2d(2)
    )
    self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
    self.drop = nn.Dropout2d(0.25)
    self.fc2 = nn.Linear(in_features=600, out_features=120)
    self.fc3 = nn.Linear(in_features=120, out_features=10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out

```

- nn.Sequential : 여러 계층을 차례로 쌓을 수 있도록 $Wx+b$ 와 같은 수식과 활성화 함수 연결
- 합성곱층 : 합성곱 연산을 통해 이미지 특징 추출
 - 합성곱이란 커널 : $n*m$ 크기의 행렬이 높이*너비 크기의 이미지를 처음부터 끝까지 훑으면서 각 원소 값끼리 곱한 후 모두 더한 값 출력
 - in_channels : 입력 채널의 수
 - 채널 : 3차원에서는 결국 깊이

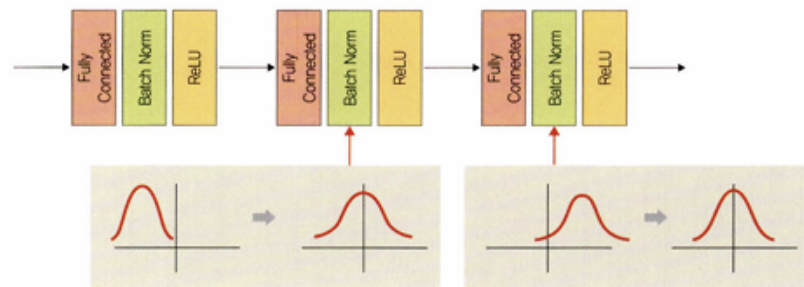
♥ 그림 5-27 채널



- out_channels : 출력 채널의 수
- kernel_size : 커널 크기를 의미 (=필터), 이미지 특징을 찾아내기 위한 공용 파라미터로, CNN에서의 학습 대상은 필터 파라미터
 - 입력 데이터를 스트라이드 간격으로 순회하며 합성곱을 계산

- BatchNorm2d : 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용하여 정규화하는 것
 - 배치단위나 계층에 따라 입력값의 분포가 모두 다르지만 정규화를 통해 분포를 가우시안 형태로 만들

♥ 그림 5-28 BatchNorm2d



- MaxPool2d : 이미지 크기를 축소시키는 용도 → 합성곱층의 출력 데이터를 입력으로 받아 출력 데이터 크기를 줄이거나 특정 데이터 강조
 - max pooling, average pooling, min pooling
 - kernel_size : m*n 행렬로 구성된 가중치
 - stride : 입력 데이터에 커널을 적용할 때 이동할 간격
- Conv2d에서 사용하는 하이퍼 파라미터값들에 따라 출력 크기가 달라지고, 최종적으로 완전 연결층(fully connected layer)로 전달됨
- Conv2d 계층에서 출력 크기 구하는 공식
 - 출력 크기 = $(W - F + 2P) / S + 1$
 - W : 입력 데이터 크기, F : 커널 크기, P : 패딩 크기, S : 스트라이드
 - ex) `nn.Conv2d(in_channels = 1, out_channels = 32, kernel_size = 3, padding = 1)`
 $\Rightarrow (784 - 3 + (2 * 1)) / 1 + 1 = 784$
`[32, 784, 784]`
- MaxPool2d 계층에서 출력 크기 구하는 공식
 - 출력 크기 : 입력필터의크기/커널 크기
 - ex) `nn.MaxPool2d(kernel_size = 2, stride = 2)`
 $\Rightarrow 784 / 2 : [32, 392, 392]$

```
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
```

```

accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)

            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total
        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".format
(count, loss.data, accuracy))

```

모델 학습 및 평가 : 실제로 이미지가 많아지면 단순 심층 신경망으로는 정확한 특성 추출 및 분류가 불가능하므로 합성곱 신경망을 생성할 수 있도록 학습해야함